APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs

André Pereira
Deloitte and Faculty of Engineering, University of Porto
Porto, Portugal
adbp@live.com.pt

Bruno Lima
LIACC, Faculty of Engineering, University of Porto
Porto, Portugal
brunolima@fe.up.pt

João Pascoal Faria
INESC TEC, Faculty of Engineering, University of Porto
Porto, Portugal
jpf@fe.up.pt

Abstract

Modern software systems rely heavily on Web APIs, yet creating meaningful and executable test scripts remains a largely manual, time-consuming, and error-prone task. In this paper, we present *APITestGenie*, a novel tool that leverages Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and prompt engineering to automatically generate API integration tests directly from business requirements and OpenAPI specifications. We evaluated *APITestGenie* on 10 real-world APIs, including 8 APIs comprising circa 1,000 live endpoints from an industrial partner in the automotive domain. The tool was able to generate syntactically and semantically valid test scripts for 89% of the business requirements under test after at most three attempts. Notably, some generated tests revealed previously unknown defects in the APIs, including integration issues between endpoints. Statistical analysis identified API complexity and level of detail in business requirements as primary factors influencing success rates, with the level of detail in API documentation also affecting outcomes. Feedback from industry practitioners confirmed strong interest in adoption, substantially reducing the manual effort in writing acceptance tests, and improving the alignment between tests and business requirements.

CCS Concepts

• **Software and its engineering → Software verification and validation**.

Keywords

API specifications, API testing, Business requirements, Large language models, OpenAPI, Test script generation

ACM Reference Format:
André Pereira, Bruno Lima, and João Pascoal Faria. 2026. APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs. In *7th ACM/IEEE International Conference on Automation of Software Test (AST 2026) (AST '26), April 13–14, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3793654.3793743

1 Introduction

Generative Artificial Intelligence (AI) has witnessed remarkable progress in recent years, reshaping the landscape of intelligent systems with applications in many domains. Studies show that intelligent assistants powered by Large Language Models (LLMs) can now generate program and test code with high accuracy based on textual prompts [19], boosting developers' and testers' productivity.

Although LLMs have been explored for several test generation tasks, there is a lack of studies exploring them for testing Web APIs, which constitute a fundamental building block of modern software systems. The rapid proliferation of APIs in recent years [12] has further heightened the need for scalable and automated testing approaches that can keep pace with rapid software evolution and deployment cycles.

Identifying and creating relevant API tests is a tedious, time-consuming task [11], requiring significant programming expertise, particularly as system complexity increases. While integration and system tests ensure functional integrity, they are more challenging to generate because the expected behavior is often described at a high-level of abstraction. Existing API test automation tools often fail to leverage high-level requirements to produce comprehensive test cases and detect semantic faults.

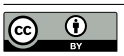
To overcome such limitations by taking advantage of recent developments in Generative AI, we present *APITestGenie* – a tool that leverages LLMs to generate executable API test scripts from business requirements written in natural language and API specifications documented using OpenAPI [14], the most widely adopted standard for documenting API services. APITestGenie was developed and validated in collaboration with an industrial partner in the automotive domain.

We evaluated *APITestGenie* on 10 real-world APIs, including 8 APIs comprising circa 1,000 live endpoints from our industrial partner. The tool was able to generate syntactically and semantically valid test scripts (requiring no manual editing) for 89% of the business requirements under test after at most three attempts.

To summarize, the main contributions of this paper are:

- A novel tool (*APITestGenie*) that leverages LLMs, retrieval-augmented generation (RAG), and prompt engineering to generate executable API test scripts from business requirements in natural language and OpenAPI specifications of Web APIs.
- Results of an experimental evaluation of the tool on APIs of varying complexity from an industrial partner in the automotive domain, obtaining up to 89% valid test scripts, a significant reduction in manual effort, and evidence of the influence of API complexity and requirements detail on test generation success.

In next sections, we analyze related work, present our solution and its evaluation, and point out conclusions and future work.



AST '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2476-3/2026/04
https://doi.org/10.1145/3793654.3793743

2 Related Work

Developers often perceive testing as a high-effort, low-reward activity. Empirical studies show that even when developers spend up to 40% of their time on testing, it is frequently seen as excessive given the perceived complexity and limited immediate value [16]. This effort is further undermined by the lack of recognition that testing receives within teams, leading developers to prioritize feature delivery over comprehensive test coverage. These factors contribute to insufficient testing practices in many software projects and motivate the need for automation tools that reduce the manual burden of test creation while increasing the value of testing within development workflows.

In recent years, several approaches and tools have emerged for testing REST APIs, with a significant rise in research since 2017, focusing on test automation using black-box techniques and OpenAPI schemas [5]. Some recent approaches also take advantage of LLMs.

In a 2022 study [9], the authors compared the performance of ten state-of-the-art REST API testing tools from both industry and academia on a benchmark of twenty open-source APIs, in terms of code coverage and failures triggered (5xx status codes), highlighting EvoMaster among the best-performing.

EvoMaster [4] can generate test cases for REST APIs in both white-box and black-box modes, with worse results in the former due to the lack of code analysis. It works by evolving test cases from an initial random population using an evolutionary algorithm, trying to maximize measures such as code coverage (only in white-sbox mode) and fault detection, using several heuristics [1]. Potential faults considered for fault finding are based on HTTP responses with 5xx status codes and discrepancies between API responses and expectations derived from OpenAPI schemas.

However, like other approaches that rely solely on OpenAPI specifications as input [6, 17, 18], EvoMaster is limited by the absence of requirements specifications and context awareness, which restricts the variety of tests that can be generated and the types of faults that can be detected. Our goal is to leverage requirements specifications and the context-awareness capabilities of LLMs to overcome these limitations, enabling the generation of more comprehensive and effective tests.

Recent works have started leveraging LLMs for REST API testing. RESTGPT [8] enhances testing by extracting rules and parameter values from OpenAPI descriptions but does not generate executable scripts. RESTSpecIT [3] uses LLMs for automated specification inference, discovering undocumented routes and query parameters, and identifying server errors. Both approaches complement ours, as the enhanced or inferred specifications can be used as input for subsequent test script generation.

LlamaRestTest [7] employs fine-tuned and quantized language models to infer inter-parameter dependencies and dynamically generate realistic inputs based on server feedback, achieving high code coverage. Its strength lies in adaptability during runtime, though it primarily relies on API specifications without explicit linkage to stakeholder requirements.

AutoRestTest [15] adopts a multi-agent reinforcement learning framework enriched with Semantic Property Dependency Graphs (SPDG), integrating LLM-driven inputs to intelligently manage operation ordering and parameter values. This approach significantly improves fault detection and test coverage, yet it still does not incorporate natural-language business requirements into its input process.

LogiAgent [20] advances beyond traditional status-code validation by introducing logical oracles through a multi-agent LLM framework. By verifying the logical correctness of responses against expected business behaviors, LogiAgent uncovers subtle domain-specific faults. However, its generation of test scenarios is based only on the API specification and does not explicitly utilize natural-language business requirements as input.

DeepREST [2] applies curiosity-driven deep reinforcement learning to implicitly uncover API constraints and execution orders. Despite its capability to expose unforeseen faults and improve coverage, it does not integrate textual stakeholder requirements or leverage natural-language contexts provided by end-users.

In contrast, our proposed approach, *APITestGenie*, uniquely distinguishes itself by explicitly incorporating natural-language business requirements alongside OpenAPI specifications through Retrieval-Augmented Generation (RAG). Thus, *APITestGenie* bridges the critical gap between user-centric business expectations and automated REST API testing, complementing other approaches.

3 Solution Design

3.1 Architecture and Workflow

APITestGenie autonomously generates, improves, and executes test cases. Its architecture (see Figure 1) is divided into three modular processes: *Test Generation*, *Test Improvement*, and *Test Execution*. While each flow can operate independently, they are designed to work together for a more complete solution.

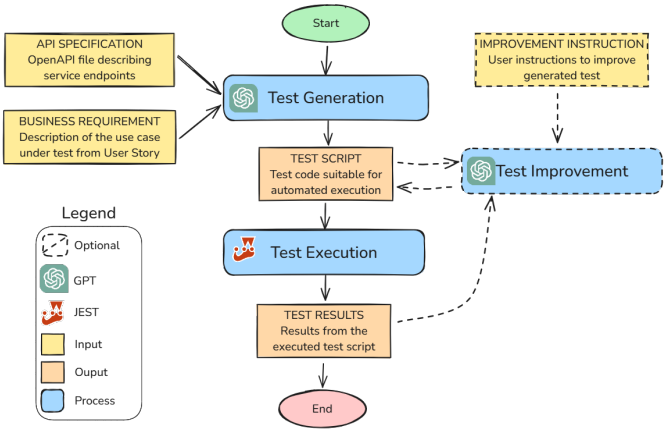


Figure 1: APITestGenie flow diagram, showcasing the main processes in the system, inputs and outputs.

The *Test Generation* process receives as input the API specification and the business requirement to generate a test script. Depending on the API size, we use either retrieval-augmented generation (RAG) for larger API specs or full processing for smaller ones. We then construct the system and user prompts based on the TELeR taxonomy [13] for a *<single turn, instruction, high detail, defined>*

use case (see Section 3.2). Lastly, the LLM is invoked and the generated content is parsed, producing the final executable test script. An example of a generated test script is included in the Appendix.

The *Test Improvement* process can be used to refine the generated script based on previous test results and user-provided feedback.

The *Test Execution* process uses the Jest framework in a TypeScript environment to execute the generated test script and report the results.

The design promotes modularity, facilitation the integration or upgrading of components.The clear separation of responsibilities among components enhances their specialization and effectiveness, ultimately improving the quality and accuracy of generated content.

3.2 System and User Prompt

The system prompt is structured as follows:

(1) *Context* — specifies the task to be performed (see Figure 2), including:
   (a) Model's role;
   (b) High-level objetive;
   (c) Test structure example;
   (d) Information on available environment variables.
(2) *Performance* — guidelines to evaluate the LLM output (see Figure 3), including:
   (a) Output evaluation metrics;
   (b) Generation guidelines.
(3) *Output* — describes the reasoning steps to generate the output (see Figure 4):
   (a) Reasoning steps;
   (b) Output components and structure.

The user prompt is structured as follows:

(1) *Requirement* — business requirement under test (usually written in the format of a user story);
(2) *API* — API specification under test (in OpenAPI).

```
As an AI coding assistant, my goal is to facilitate the
     creation of executable API integration tests in
     TypeScript.
Many users may not be familiar with coding, so I am here to
     bridge the gap and help them craft tests that validate
     their application's business requirements.

To ensure the tests are practical and meet the users' needs,
      I will generate integration test cases in TypeScript
     using the Axios and Jest libraries.
These tests will be designed for immediate execution and
     will interact with the API endpoints defined in the API
      specification.

I am expected to create more complex tests following the
     example
***
{test_example}
***
```

Figure 2: "Context" section of the system prompt.

```
TESTS WILL BE ASSESSED ON SEVERAL KEY FACTORS:
- Executability: The tests must run smoothly and without
     errors.
- Relevance: The tests must be meaningful and appropriate
     for the requirement.
- Correctness: The test code should be free of logical
     errors.
- Coverage: The tests should cover as many endpoints and
     scenarios as possible to ensure thorough validation.
- Code Quality: The code is reliable, executable, and
     includes clear explanations where necessary.
- Endpoint Accuracy: The API call matches the type of the
     request and response object.

GUIDELINES FOR TEST GENERATION:
- Informational Completeness: Collect as much information as
      possible. Additionally, generate experimental data to
     use in placeholders.
- Environment Setup: Ensure all necessary data is collected
     to guarantee that the test can execute correctly.
```

Figure 3: "Performance" section of the system prompt.

````
Here's how the test generation will be accomplished:
\\
1. CLARIFYING THE BUSINESS REQUIREMENT:
- Summarize the business requirement and explain what
     aspects the tests verify.
- Describe the data and environment state of the test.

2. LISTING ENDPOINTS:
- List the endpoints that the test script will interact with
     .
- For each endpoint, specify the types of the request and
     response objects.
- Provide a brief list of steps to reach the correct test
     environment state.

3. CRAFT EXECUTABLE TEST CODE:
- Generate test cases in TypeScript using Axios and Jest.
- Present the code in a single block, ready to run.
- Explain unclear properties and minimize extraneous prose.

Document your generation using the following format:

REQUIREMENT:
<1. **Clarifying the Business Requirement**>
ENDPOINTS:
<2. **Listing Endpoints**>
TEST:
```typescript
<3. **Craft Executable Test Code**>
````

Figure 4: "Output" section of the system prompt.
````

3.3 API Pre-processing and Retrieval-Augmented Generation

A significant limitation of current LLM models is their context window size. To mitigate this, we first created a preprocessing script that simplifies the raw OpenAPI specification by removing image tags and administrative or deprecated resources. This approach reduces token usage while keeping the essential content. Although effective for smaller APIs, this method proved insufficient for larger ones, motivating the adoption of a more robust technique.

To overcome the limitations of LLM context windows when dealing with large API specifications, we implemented a Retrieval-Augmented Generation (RAG) [10] pipeline. This process allows selective retrieval of the most relevant parts of an API specification to include in the prompt provided to the LLM. The pipeline begins by preprocessing the OpenAPI specification and splitting it into semantically coherent chunks ranging from 800 to 1200 tokens, preserving contextual consistency while maintaining conciseness. Each chunk is then embedded using a text-embedding model, and the resulting vectors, along with their documents, are stored in a vector database.

To retrieve relevant content for a given business requirement, we use a multi-step query process. The requirement is first expanded using an LLM-based script that rephrases it into multiple semantically similar variants, capturing different perspectives of the same intent. Each expanded version is embedded using the same model and used to query the vector database. The retrieved chunks are then aggregated to form a consolidated set of relevant documents, which are appended to the system prompt for test generation. Because retrieval is based on multiple semantically equivalent versions, the final context is robust and less susceptible to hallucinations.

This RAG-enhanced prompt construction enables *APITestGenie* to scale effectively to large and complex APIs without exceeding the LLM's input limitations, while maintaining alignment with the business logic expressed in user requirements. For a visual representation of the flow with RAG, see Figure 5.

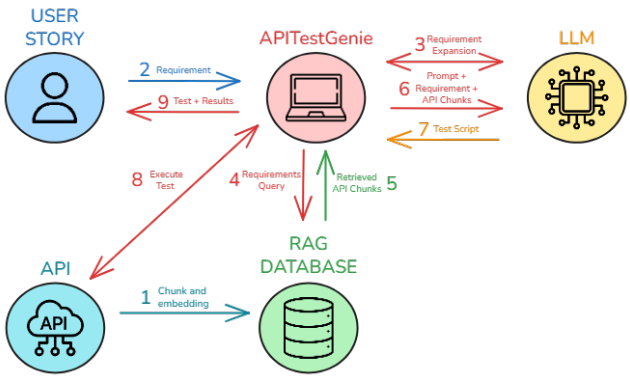


Figure 5: APITestGenie's RAG process builds a database from the API specification, expands requirements, and retrieves relevant API segments to give the LLM precise context for generating targeted test scripts.

4 Evaluation

We evaluated *APITestGenie* in two phases. First, we conducted an experimental evaluation on ten APIs (including eight from our industrial partner in the automotive domain), collecting metrics on correctness, generation time, and cost of the generated test scripts. Second, we organized a practical workshop with fifteen experts from our industrial partner to gather feedback on the tool's capabilities and adoption. We also compared *APITestGenie* with EvoMaster (a state-of-the-art tool) for one of the public APIs tested.

The main research questions we want to answer are:

- **RQ1** — How effectively and efficiently can *APITestGenie* generate API test scripts from business requirements and API specifications?
- **RQ2** — How do API complexity, API documentation detail and requirements detail influence the effectiveness of test script generation by *APITestGenie*?
- **RQ3** — How do industry practitioners perceive the usefulness and adoptability of *APITestGenie* for API test script generation?

4.1 Experimental Evaluation

4.1.1 Materials and methods. We assessed the robustness of test scripts generated by *APITestGenie* for ten APIs with varying complexity and documentation levels (see Table 1).

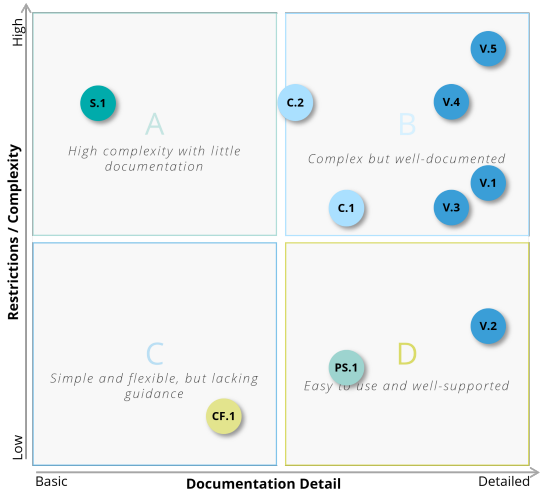


Figure 6: Characterization of the APIs under test according to their internal complexity and documentation detail.

To help answer RQ2, we classified the APIs under test along two dimensions (see Figure 6):

- Documentation detail – considering whether API methods are properly documented, examples of calls are provided, possible errors are listed, and examples of input data (parameters) and expected outputs (response schemas) are included;
- API complexity – considering the number of available endpoints, the relationships and dependencies among them, the presence of authentication requirements, and the predictability of API responses and errors.

Table 1: Experimental test generation results for 10 APIs and 25 business requirements.

API	Access	Original tokens[a]	Simplified tokens[b]	Business Req. (BR)	Valid Test Scripts (TS)[c]	BR with ≥1 valid TS	Generation Time (avg.)
CF.1 Cat Fact[e]	Public	754	754	1	2	1	92s
PS.1 Pet Shop[e]	Public	4,070	4,070	2	4	2	75s
V.2 Vehicle Configuration	Private	430,739	16,094	5	15	5	102s
Subtotal Low API Complexity				**8**	**21 (87.5%)**	**8 (100%)**	
C.1 Customer Authorization	Private	25,966	25,475	2	5	2	113s
C.2 Customer Account	Private	47,497	47,004	2	5	2	98s
S.1 Web store	Private	49,761	49,761	2	3	1	154s
V.1 Vehicle Accessories	Private	882,272	14,018	2	3	1	115s
V.3 Vehicle Localization	Private	1,620,488	82,267	2	3	2	131s
V.4 Vehicle Production	Private	417,109	108,724[f]	4	8	4	146s
V.5 All Vehicle Services	Private	5,021,555	424,465[f]	3	4	2	227s
Subtotal High API Complexity				**17**	**31 (60.8%)**	**14 (82.4%)**	
Total		**9,254,211**	**772,632(8.3%)**	**25**	**52 (69.3%)**	**22 (88.6%)**	**126s**

[a]Number of tokens in the original OpenAPI specification counted with the tiktoken tokenizer.

[b]Number of tokens in the simplified spec (used as input for test generation), after removing irrelevant tags and resources.

[c]Number of syntactically and semantically valid test scripts in three generation attempts per business requirement.

[d]RAG was used to filter simplified API specs with over 100K tokens, reducing them to fit the LLM's maximum prompt size.

[e]Public APIs available at https://catfact.ninja/ and https://petstore.swagger.io/.

```
1. Basic BR description (user story):
As a user, I want to receive a new and random cat fact every
     time I open the app or refresh the content so that I
    can learn intriguing information about cats and stay
    engaged with the app.

2. BR including concrete data:
As a user, I want to retrieve all available models in
    Germany so that I can know which products are available
    .

3. BR including procedural information:
As a user, I want to retrieve the price of accessories for a
     product in a given market. First, use the product
    service to get a list of all available models. Then,
    use the accessories service to retrieve available
    accessories, and finally use the pricing service to
    calculate their prices.
```

Figure 7: Examples of business requirements (BR) with varying detail.

Three of the APIs are low-complexity, with two of them being public APIs (CF.1 Cat Fact and PS.1 Pet Shop) and the other private. The other seven private APIs from our industrial partner in the automotive domain have a higher level of complexity, comprising approximately 1,000 live endpoints.

Overall, we used 25 unique business requirements (BR) as input for test script (TS) generation. The BR were usually described in the form of user stories, with varying levels of detail (see Figure 7).

To improve statistical significance and minimize the effect of LLM randomness on the results, we performed three independent test script generation attempts per BR, totalling 75 generation attempts.

The model used in the experiment was GPT-4-Turbo, featuring a 128k-token context window, deployed under an enterprise configuration with data privacy safeguards.

We then executed the generated test scripts to check their syntactic validity and manually inspected the executable ones to determine their semantic validity. By a *valid test script*, we mean a syntactically and semantically valid test script, i.e., a test script that executes without syntax errors and is considered semantically accurate in manual inspection (confirming or refuting the business requirement when run against the target API).

4.1.2 Overall performance results. As shown by the totals in Table 1, 52 out of 75 generation attempts (three per business requirement) resulted in valid test scripts, corresponding to an overall success rate of 69.3%. In total, 88.6% of the business requirements had at least one valid test script generated within three attempts.

Average time and cost were 126s and 0.37€ per generation, respectively, with a tendency to decrease as newer LLMs are adopted.

These results indicate that *APITestGenie* can effectively generate executable and functionally meaningful test scripts from business requirements and API specifications.

4.1.3 Assessment of the generated test scripts. Table 2 presents the results of the assessment of the generated test scripts.

Out of 75 generation attempts, 4 (5.3%, *a* and *b*) resulted in empty or syntactically invalid scripts, which can be automatically identified and typically resolved by regeneration.

Table 2: Assessment of the generated test scripts, distinguishing valid and invalid test scripts.

Result Type	Test Scripts	Description	Resolution
Invalid:	**23 (30.7%)**		
a) Empty script [a]	1	The generated script is empty.	Re-generate.
b) Syntax error [a]	3	The script cannot be executed due to syntax errors.	Re-generate.
c) Semantic error(s)	19	Test execution failed due to LLM hallucinations: wrong/non-existent import, attribute, operation, response structure validation, etc.	Fix script, re-generate, or run improvement flow (including error information).
Valid:	**52 (69.3%)**		
d) Pass	31	Test execution succeeds, and the script is considered semantically valid in manual inspection.[b]	Save script for regression testing.
e) Fail: API defect	12	Test execution fails due to inconsistencies between the API specification or implementation and the expected behavior described in the requirements. Failures include inconsistencies with requirements in tests involving multiple endpoints.	Fix specification or implementation and re-run.
f) Fail: Insufficient documentation	2	Test execution fails because key information is missing in the API specification.	Fix specification and re-generate.
g) Fail: Environment issue	7	Test execution fails due to missing or incorrect environment variables (e.g., access tokens).	Fix environment setup and re-generate.

[a]These errors are autonomously identified (non-executable), enabling automatic reattempt.

[b]In our experiment, all scripts that executed successfully and passed were considered valid in manual inspection.

The remaining scripts were executed, their test results collected, and the scripts manually inspected for semantic validity.

In 19 cases (25.3%, *c*), test execution failed due to semantic errors. In practice, such cases of LLM hallucination can be resolved by minor fixes, regeneration, or by including error information in the improvement prompt.

In 31 cases (41.3%, *d*), test execution succeeded and the scripts were considered semantically valid.

The remaining 21 cases (28%) corresponded to semantically valid scripts whose execution failed due to API defects (12 cases, 16%), insufficient API documentation (2 cases, 3%), or environment issues (7 cases, 9%).

These results highlight the defect-detection capability of the generated tests (19% revealed problems in the API implementation or documentation) and the potential to reduce manual (re)writing effort (only 19 out of the 71 inspected scripts might require minor fixes).

The Appendix includes an example script that interacts with two endpoints and uncovers an API defect that would be difficult to detect with unit tests.

4.1.4 Analysis of influencing factors. We investigated whether API complexity, API documentation detail, and business requirements detail had a statistically significant impact on the outcomes. For each group, we compared the number of valid scripts to the total number of generation attempts (three per requirement) using the chi-squared test.

API complexity showed a statistically significant effect. Low-complexity APIs had a substantially higher proportion of valid scripts (87.5% in Table 1) than high-complexity APIs (60.8% in Table 1), with a p-value of 0.038 in the chi-squared test. In contrast, documentation detail ($p = 0.57$) did not show a significant effect.

Regarding the impact of the level of detail in the business requirements, we observed that requirements including concrete data (6 BRs) led to a higher proportion of valid test scripts (88.9%) compared to those without concrete data (19 BRs, 63.2% success rate). The impact on test generation success rate was statistically significant, with a p-value of 0.039 in the chi-squared test. The inclusion of procedural information did not show a statistically significant impact.

These findings suggest that, while improving documentation may help, reducing API complexity (e.g., by decomposing large APIs into smaller services) and providing concrete data in the business requirements had the greatest measurable impact in our experiments.

4.2 Comparison with EvoMaster

To contextualize the performance of APITestGenie, we compared it with EvoMaster [4], one of the most established tools for automated REST API test generation. EvoMaster was executed in black-box mode against the same Pet Store API used in our example test, see Appendix, Listing 1. Both tools were limited to a 70 s execution/generation window, which corresponds approximately to the average time required for APITestGenie to produce a test script.

EvoMaster generated 27 test cases in total. Of these, **22 (81%)** triggered failures, **2 (7%)** executed successfully, and **3 (11%)** required manual validation. According to EvoMaster's internal metrics, 19 of the reported failures were due to structural mismatches between

the received responses and the OpenAPI schema *"Received a Response From API With a Structure/Data That Is Not Matching Its Schema"*, and one failure corresponded to *"HTTP Status 500"*. Under our evaluation framework, such schema-mismatch errors would be classified as documentation defects, while the 500 status case, caused by malformed input data, would be considered an invalid test case resulting from a hallucination.

From a functional perspective, EvoMaster successfully detects schema and documentation inconsistencies but focuses on testing each endpoint independently. In contrast, APITestGenie generates multi-endpoint integration tests directly derived from business requirements. For the Pet Store scenario, APITestGenie produced a two-step test flow that (i) creates a pet using `POST /v2/pet` and (ii) retrieves it using `GET /v2/pet/{petId}`. The generated test revealed that the retrieved pet differed from the one created, an integration-level defect that EvoMaster, which treats endpoints in isolation, cannot detect.

In summary, EvoMaster achieved endpoint-level structural validation and uncovered documentation defects efficiently, whereas APITestGenie provided requirement-driven, end-to-end validation capable of exposing logical or cross-endpoint faults. These results highlight the complementary nature of the two tools: EvoMaster excels at structural conformance and schema testing, while APITestGenie extends coverage to business-level behaviors and system integration flows.

4.3 User Feedback

We conducted a hands-on *APITestGenie* workshop with technical staff from our industrial partner in the automotive domain, who provided informed feedback on the relevance and completeness of the generated tests. During the workshop, we created four unique test scripts, demonstrating the tool's ability to handle progressively more complex requirements and APIs. Out of 15 participants, 11 responded to an anonymous survey, the results of which are shown in Figure 8.

Overall, *APITestGenie* was well received, with most participants finding the tests complete and relevant and expressing interest in daily use. Some participants noted the need for improvements in data security and in integration with existing testing environments.

All respondents agreed or strongly agreed that *APITestGenie* can accelerate the test generation process compared to a manual approach. To better understand the potential gains, we conducted follow-up discussions with several workshop participants, who noted that manually scripting multi-endpoint API integration tests typically takes 20–30 minutes per case. In contrast, *APITestGenie* reduces this to approximately two minutes per test, with an additional 2–5 minutes for manual review, yielding a 4–5× reduction in effort.

These results suggest substantial return on investment in large-scale enterprise projects, particularly when paired with strong documentation and integration into existing workflows. By automating boilerplate test creation, *APITestGenie* enables developers and QA engineers to focus on higher-value tasks such as handling edge cases and improving robustness.

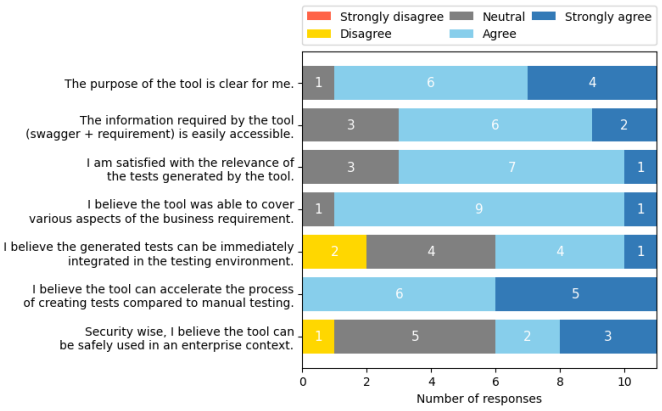


Figure 8: Opinions on *APITestGenie*-generated tests from 11 practitioners with over five years of domain experience, collected anonymously after a workshop.

4.4 Answers to Research Questions

RQ1: In our experiments, *APITestGenie* effectively generated executable API test scripts from business requirements and API specifications. Across 75 generation attempts, 69.3% of the scripts were syntactically and semantically valid, and at least one valid script was produced for 88.6% of the business requirements within three attempts. The generation process was efficient, averaging 126 seconds per script, and required minimal human intervention.

RQ2: In our experiments, API complexity and level of detail in business requirements significantly influenced test generation effectiveness. Low-complexity APIs yielded more valid scripts (87.5%) than high-complexity ones (60.8%; $p = 0.038$), and requirements with concrete data achieved higher success rates (88.9%) than those without (63.2%; $p = 0.039$). Documentation detail showed no significant effect ($p = 0.57$). Overall, structural simplicity and explicit data examples were stronger predictors of successful automated test generation than documentation thoroughness.

RQ3: Industry practitioners perceived *APITestGenie* as both useful and adoptable for API test generation. All survey respondents agreed that it accelerates test creation compared to manual scripting, and most found the generated tests relevant and complete. Participants expressed interest in using the tool in daily practice, noting that it could substantially reduce repetitive work. However, they also identified adoption barriers, including data security concerns when using external LLMs and the need for seamless integration with existing testing workflows. Overall, the feedback indicates a positive perception of *APITestGenie*'s usefulness and practicality, contingent on addressing these integration and security requirements.

4.5 Limitations and Threats to Validity

This study has several limitations and potential threats to validity that should be considered when interpreting the results.

4.5.1 Internal validity. Manual inspection, while systematic, introduces some subjectivity in assessing semantic validity. The analysis of influencing factors relied on naturally occurring variations in the

available APIs and business requirements, which were not experimentally controlled, limiting causal inference. Nevertheless, this reflects the realities of industrial settings, where variables such as API complexity and documentation detail cannot be artificially balanced. To enhance internal validity, we applied consistent evaluation criteria, performed repeated generation attempts, and cross-checked ambiguous assessments among the authors. These measures improve reliability while acknowledging the inherent limitations of an observational design.

4.5.2 External validity. Our evaluation involved ten APIs, eight of which came from a single industrial partner in the automotive domain. Consequently, results may not generalize to APIs in other domains, with different design styles or documentation practices. However, using industrial-grade APIs and experienced practitioners increased the practical validity of the study and provided realistic insights into tool applicability in production-like environments. User feedback was collected from 11 practitioners with over five years of experience, offering informed perspectives but limiting broader generalizability. The study compared *APITestGenie* to manual test creation but not to other automated testing tools, as most existing approaches rely on API specifications, models, or execution traces rather than natural-language business requirements. Future studies should replicate the evaluation across additional domains and include comparative experiments with alternative automated approaches as they emerge to better position *APITestGenie* within the broader landscape.

4.5.3 Construct validity. The evaluation metrics captured generation effectiveness through the proportion of valid scripts and the number of business requirements with at least one valid script. While informative, the latter may overestimate practical usability, due to the manual review effort. On the other hand, the metrics do not reflect adaptive improvements that could arise if feedback from failed executions were used in subsequent prompts. Although the *Test Improvement* flow in *APITestGenie* supports such refinement, it was intentionally disabled to ensure comparability across the three independent attempts per requirement. Future work could enable this flow or add automatic regeneration after syntax or runtime errors to assess the benefits of self-correction and reduce manual review effort.

5 Conclusion

In this work, we presented *APITestGenie*, an AI-driven tool for generating executable API test scripts from business requirements and OpenAPI specifications. Our approach leverages LLMs and RAG to scale testing in systems where traditional automation methods fall short, especially in complex, multi-endpoint integration scenarios.

Evaluated in collaboration with an industrial partner in the automotive domain, *APITestGenie* demonstrated strong results: up to 89% script validity with minimal manual intervention, and generation times fast enough to support agile testing workflows. The tool was well received by practitioners, who reported productivity gains and improved requirement traceability.

Importantly, our findings highlight that successful industrial adoption of GenAI tools depends not only on model performance but also on API complexity, documentation quality, clarity of business requirements, and seamless integration into development pipelines. While human validation remains necessary for critical testing paths, *APITestGenie* already offers a valuable assistant to accelerate QA cycles.

Our evaluation targets the generation of integration tests from business requirements, which is distinct from most related tools that focus primarily on single-endpoint testing or rely exclusively on API specifications. By leveraging business requirements as input, *APITestGenie* generates executable tests that simulate realistic, multi-endpoint scenarios, providing broader validation of service interactions and data flow. Although this focus means our results are not directly comparable with those of existing tools, *APITestGenie* complements prior approaches by enabling comprehensive system-level testing that aligns more closely with actual business needs, as demonstrated in the comparison conducted with EvoMaster for one of the public APIs tested.

Looking forward, we plan to integrate the tool into CI/CD pipelines, address enterprise security considerations for LLM use, explore agent-based extensions, and conduct further evaluation studies. We believe that *APITestGenie* represents a practical and scalable pathway to GenAI-enabled testing in real-world software engineering.

6 Data and Code Availability

To support reproducibility and future research, whilst complying with the double-blind review policy, an anonymized repository containing a simplified version of the source code, example test scripts, and representative prompts used in this study is available at the folloinw url: https://github.com/Andrepereira2001/ApiTestGenie.

References

[1] Andrea Arcuri. 2017. RESTful API Automated Test Case Generation. In *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. 9–20. doi:10.1109/QRS.2017.11

[2] Davide Corradini, Zeno Montolli, Michele Pasqua, and Mariano Ceccato. 2024. Deeprest: Automated test case generation for rest apis exploiting deep reinforcement learning. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1383–1394.

[3] Alix Decrop, Gilles Perrouin, Mike Papadakis, Xavier Devroey, and Pierre-Yves Schobbens. 2024. You Can REST Now: Automated Specification Inference and Black-Box Testing of RESTful APIs with Large Language Models. *arXiv preprint arXiv:2402.05102* (2024).

[4] EvoMaster. [n. d.]. EvoMaster: A Tool For Automatically Generating System-Level Test Cases. [Online]. Available: www.evomaster.org.

[5] Amid Golmohammadi, Man Zhang, and Andrea Arcuri. 2023. Testing RESTful APIs: A Survey. *ACM Transactions on Software Engineering and Methodology* 33, 1 (2023), 1–41.

[6] Stefan Karlsson, Adnan Čaušević, and Daniel Sundmark. 2020. QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 131–141. doi:10.1109/ICST46399.2020.00023

[7] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. Llamaresttest: Effective rest api testing with small language models. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)* 2, FSE (2025), 465–488.

[8] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. 2024. Leveraging Large Language Models to improve REST API testing. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 37–41.

[9] Myeongsoo Kim, Qi Xin, Saurabh Sinha, and Alessandro Orso. 2022. Automated test generation for REST APIs: No time to rest yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 289–301.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel,

Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[11] Macario Polo, Pedro Reales, Mario Piattini, and Christof Ebert. 2013. Test Automation. *IEEE Software* 30, 1 (2013), 84–89. doi:10.1109/MS.2013.15

[12] Postman. 2023. *2023 State of the API report.* Technical Report. Postman.

[13] Shubhra Kanti Karmaker Santu and Dongji Feng. 2023. TELeR: A General Taxonomy of LLM Prompts for Benchmarking Complex Tasks. arXiv:2305.11430

[14] SmartBear Software. [n. d.]. OpenAPI Specification. [Online]. Available: https://swagger.io/specification/.

[15] Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. Autoresttest: A tool for automated rest api testing using llms and marl. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 21–24.

[16] Philipp Straubinger and Gordon Fraser. 2023. A Survey on What Developers Think About Testing. arXiv:2309.01154 [cs.SE] https://arxiv.org/abs/2309.01154

[17] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2023. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. arXiv:2307.00588

[18] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTESTGEN: Automated Black-Box Testing of RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 142–152. doi:10.1109/ICST46399.2020.00024

[19] Michel Wermelinger. 2023. Using GitHub Copilot to Solve Simple Programming Problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (<conf-loc>, <city>Toronto ON</city>, <country>Canada</country>, </conf-loc>) *(SIGCSE 2023)*. Association for Computing Machinery, New York, NY, USA, 172–178. doi:10.1145/3545945.3569830

[20] Ke Zhang, Chenxi Zhang, Chong Wang, Chi Zhang, YaChen Wu, Zhenchang Xing, Yang Liu, Qingshan Li, and Xin Peng. 2025. LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents. *arXiv preprint arXiv:2503.15079* (2025).

A Example of Test Script

Listing 1 presents an example of a generated test script interacting with the `/Post` and `/Get` endpoints to create a pet and retrieve it by ID. The execution of the test exposes a defect in the PetShop API, as the retrieval step fails.

Listing 1: Example of a generated test script interacting with the `/Post` and `/Get` endpoints.

```
import axios from 'axios';

describe('Adding a New Pet to the Store', () => {
  const baseUrl = 'https://petstore.swagger.io/v2';
  let createdPetId;

  test('should create a unique pet', async () => {
    const newPet = {
      name: 'Fluffy',
      photoUrls: ['http://example.com/photo1'],
      category: { id: 1, name: 'Dogs' },
      tags: [{ id: 1, name: 'Friendly' }],
      status: 'available'
    };

    const response = await axios.post(`${baseUrl}/pet`, newPet, {
      headers: {
        'Content-Type': 'application/json'
      }
    });

    // Assuming the response contains the created Pet object
    expect(response.status).toBe(200);
    expect(response.data).toHaveProperty('id');
    createdPetId = response.data.id; // Store the ID for further use
    expect(response.data.name).toBe('Fluffy');
  });

  test('should retrieve the created pet by ID', async () => {
    if (!createdPetId) {
      throw new Error('Failed to store the created pet ID');
    }

    const response = await axios.get(`${baseUrl}/pet/${createdPetId
      }`, {
      headers: {
        'Content-Type': 'application/json'
      }
    });

    expect(response.status).toBe(200);
    expect(response.data.id).toBe(createdPetId);
    expect(response.data.name).toBe('Fluffy');
  });
});

```